FAETERJ-PETRÓPOLIS · SISTEMAS OPERACIONAIS II · TURMA 2026/2 (NOITE)

Mandelbrot PPM

Programação paralela: uma aplicação sequencial transformada em versões com processos e com threads

Elton Giglio Kasesky
Frederico Bazano da Cruz

Professor: Douglas Ericson



O que é o conjunto de Mandelbrot?

- Cada ponto do plano é um **número complexo** `c = a + b·i`: `a` é o eixo horizontal, `b` o vertical.
- Para cada `c`, gera-se uma sequência: $z_0 = 0$, $z_{n+1} = z_n^2 + c$
- `c` **pertence ao conjunto** se essa sequência fica limitada, ou seja, nunca vai para o infinito.
- **Critério de fuga:** se em algum momento `|z| > 2`, a sequência certamente diverge. Dá para parar ali: o ponto está **fora**.
- Na prática, se não escapar em `MAX_ITERATIONS` (1000), considera-se que está **dentro**.
- A borda é um **fractal**, com detalhe infinito. **Perto dela, a sequência demora muito para escapar**, e é isso que torna o custo dos pixels tão desigual.

`c`	sequência $z_1, z_2, z_3, \ldots$	resultado
0	0, 0, 0, …	dentro
−1	−1, 0, −1, 0, …	dentro (oscila)
i	i, −1+i, −i, −1+i, …	dentro (oscila)
1	1, 2, 5, …	fora em **3** iterações
0,5	0,5; 0,75; 1,06; 1,63; 3,15	fora em **5**
0,3	0,3; 0,39; 0,45; 0,50; …	fora em **12**
0,26	0,26; 0,33; 0,37; 0,40; …	fora em **30**
0,251	0,251; 0,31; 0,35; 0,37; …	fora em **97**
0,25	aproxima-se de 0,5	dentro (ponto da borda)

Quanto mais perto da borda (aqui, de 0,25), mais iterações até decidir.

Da matemática ao pixel

- **Pixel → ponto `c`:** a imagem é uma "janela" sobre o plano complexo. Cada pixel `(x, y)` vira um ponto `c` dessa janela.
- **Iterações → cor** (escape time): conta-se quantas iterações `z` levou para escapar.
  - escapa rápido (longe do conjunto): **escuro**;
  - escapa devagar (perto da borda): **claro**, o efeito "neon";
  - nunca escapa (dentro do conjunto): **preto**.
- **Coloração suave:** `i + 1 − log₂(log₂|z|)` acrescenta à contagem uma parte fracionária, que mede o quanto `z` passou do raio. Assim a cor varia continuamente, sem faixas.
- Resultado: um tom de cinza por pixel, guardado como RGB (3 bytes).

Pixel (x, y)	Ponto c
canto superior esquerdo (0, 0)	≈ −2,67 + 1,2i
canto inferior direito (7679, 4319)	≈ 1,60 − 1,2i
janela vista	real de −2,67 a 1,60 imaginário de −1,2 a 1,2

O conjunto ocupa o real de −2 a ≈0,47 e o imaginário de −1,12 a 1,12: a janela enquadra o conjunto inteiro com margem.

Formato de imagem PPM (P6)

- **Simples:** um cabeçalho em texto (`P6`, largura, altura, valor máximo 255) seguido dos bytes RGB crus, linha a linha, 3 bytes por pixel. Sem compressão.
- **Portátil:** formato aberto do Netpbm, lido por visualizadores e editores comuns (GIMP, ImageMagick, IrfanView…).
- **Sem bibliotecas externas:** basta `stdio.h`. Não há dependência para instalar, nem no container.
- A `struct Color` é `packed` (exatamente 3 bytes, sem preenchimento), então o framebuffer na memória já tem o **mesmo layout do arquivo**: a imagem inteira é gravada com **um único `fwrite`**.
- Custo: arquivo grande (≈ 95 MB em 8K), mas a gravação leva só alguns centésimos de segundo.

```
typedef struct {
    unsigned char r, g, b;
} __attribute__((packed)) Color;

void writeImage(Color *frameBuffer, int width,
                int height, const char *outputFile) {
    FILE *file = fopen(outputFile, "wb");
    if (!file) { perror(" ... "); return; }

    // cabeçalho em texto
    fprintf(file, "P6\n");
    fprintf(file, "%d %d\n", width, height);
    fprintf(file, "255\n");

    // pixels: bytes RGB crus, de uma vez só
    size_t totalPixels = (size_t)width * height;
    fwrite(frameBuffer, sizeof(Color), totalPixels, file);

    fclose(file);
}
```

Pixel shader: do pixel ao ponto `c`

- Função que calcula a cor de **um pixel** só a partir das suas coordenadas, como um **pixel shader** de GPU. Por isso os pixels são independentes.
- **Inversão do eixo y:** na imagem, y cresce para baixo; no plano, o eixo imaginário cresce para cima.
- **Panning:** desloca a janela em 1/8 da largura para centralizar o conjunto, que fica mais à esquerda (real de −2 a 0,47).
- **Normalização 0 a 1:** `u = x / width`, `v = y / height`, independente da resolução.
- **Normalização −1 a 1:** `2u − 1`, centralizando a origem.
- **Aspect ratio:** o eixo x é multiplicado por 16/9, para o pixel ser quadrado e o fractal não ficar achatado.
- **Zoom 1,2:** amplia a janela em 20%, para o conjunto caber com margem.
- Comum às três versões, em `comum/mandelbrot.c`, junto com `renderRow` (uma linha), `writeImage` e `getTimeSec`.

```
Color pixelShader(int x, int y, int width, int height) {
    double aspectRatio = (double) width / height;
    y = height - y;              // inverte o eixo y
    x = x - width / 8;           // panning horizontal
    double u = ((double) x / width);    // 0 a 1
    double v = ((double) y / height);
    // -1 a 1, aspect ratio e zoom: c = cx + cy·i
    double cx = (((2.0f * u) - 1.0f) * aspectRatio) * 1.2;
    double cy = ((2.0f * v) - 1.0f) * 1.2;

    double zx = 0, zy = 0;       // z = 0
    for (int i = 0; i < MAX_ITERATIONS; i++) {
        double temp_zx = zx*zx - zy*zy + cx;   // z = z² + c
        double temp_zy = 2*zx*zy + cy;
        zx = temp_zx;  zy = temp_zy;
        double squaredDistance = zx*zx + zy*zy;
        if (squaredDistance > 4.0) {           // |z| > 2
            double smooth = i + 1.0
                - log2(log2(sqrt(squaredDistance)));
            return colorf(smooth / 100.0, ...);
        }
    }
    return colorf(0.0, 0.0, 0.0);            // dentro: preto
}
```

Entrada e versão sequencial

Entrada: o parâmetro `factor`

- A resolução é `16·factor × 9·factor`: sempre **16:9 exato** e sempre **inteira**.
- Com uma largura qualquer, a altura poderia sair fracionária (1000 × 562,5). Com o `factor`, não: 120 → 1920 × 1080 (Full HD), 240 → 3840 × 2160 (4K), **480 → 7680 × 4320 (8K)**.
- Um único número controla a carga de trabalho, que cresce com `factor²`. Nos testes: `factor = 480`, cerca de **33 milhões de pixels**.

Versão sequencial

- Um único fluxo calcula as linhas de cima para baixo e depois grava a imagem.
- Tempo total do programa: **11,21 s**, dos quais 11,18 s de cálculo e 0,026 s de gravação; o restante é alocação e liberação de memória. É a referência para o speedup e para a imagem.

```
int width  = 16 * factor;
int height =  9 * factor;

for (int y = 0; y < height; y++)
    renderRow(frameBuffer, y, width, height);

writeImage(frameBuffer, width, height, arquivo);
```

Demonstração:

```
./bin/sequencial out/sequencial.ppm 480
```

Estratégia de paralelização: como dividir o trabalho?

- Renderizar é **naturalmente paralelizável**: cada pixel depende só das próprias coordenadas.
- Mas o **custo de cada pixel varia muito**: pontos longe do conjunto escapam em poucas iterações; pontos dentro dele vão até 1000. O laço tem desvios (branching) e número de iterações variável.

Divisão	Granularidade	Balanceamento	Sincronização
Setores contínuos	N faixas	ruim: a faixa central concentra o conjunto	nenhuma
Round-robin de linhas	linhas intercaladas, fixas	bom, em média	nenhuma (sem coordenação entre trabalhadores)
Pixel a pixel (fila)	33 milhões de tarefas	ótimo	uma por pixel: overhead enorme frente ao trabalho de um pixel
Fila de linhas (escolhida)	4320 linhas	adapta-se sozinho	uma por linha: 4320 no total, desprezível

Fila de trabalho dinâmica, linha a linha

- A "fila" é um contador compartilhado: `next_row`, a próxima linha ainda não calculada.
- Cada trabalhador repete: **pegar uma linha → calcular → pegar a próxima**, até a fila acabar. Quem pega linhas leves processa mais linhas, e todos terminam quase juntos.
- **Condição de corrida evitada:** `next_row++` não é uma operação única. São três passos: **ler**, **somar 1** e **gravar**. Sem exclusão mútua, dois trabalhadores podem se intercalar:

Trabalhador A	Trabalhador B	`next_row`
lê 10		10
	lê 10	10
grava 11, calcula a linha 10		11
	grava 11, calcula a linha **10 de novo**	11

Resultado: trabalho duplicado e um incremento perdido. Com o lock, ler e incrementar viram uma operação indivisível, e cada linha é entregue a **exatamente um** trabalhador.

```
for (;;) {
    lock();                    // mutex ou semáforo
    int y = next_row++;        // região crítica
    unlock();

    if (y ≥ height) break;  // fila vazia

    renderRow(frameBuffer, y, width, height);
}
```

Escrita sem lock: como cada linha tem um único dono, as escritas no framebuffer nunca se sobrepõem.

Implementação com processos

- O pai lê o número de processos da linha de comando e cria os filhos com `fork()`.
- Cada filho executa o laço da fila e termina com `_exit(0)`.
  - `_exit`, e não `exit`: evita que cada filho descarregue de novo o buffer do `printf` herdado do pai.
- O pai espera todos com `waitpid()`, sem deixar zumbis, e só então grava a imagem.
- Os processos não compartilham memória por padrão: framebuffer e fila precisam ser **explicitamente compartilhados**.

```
for (p = 0; p < num_processes; p++) {
    pids[p] = fork();
    if (pids[p] == 0) {          // filho
        /* laço da fila */
        _exit(0);
    }
}
for (p = 0; p < num_processes; p++)
    waitpid(pids[p], NULL, 0);   // pai

writeImage(frameBuffer, ...);
```

Memória compartilhada

- Duas regiões criadas com `mmap(MAP_SHARED | MAP_ANONYMOUS)` **antes do fork**, herdadas por todos os filhos:
  - **framebuffer**: a imagem (cerca de 95 MB em 8K), escrita pelos filhos e gravada pelo pai;
  - **SharedState**: a fila (`next_row`) e o semáforo.
- Sem `MAP_SHARED`, o fork daria a cada filho uma **cópia própria** (copy-on-write): o pai receberia a imagem vazia e cada filho teria sua própria fila.
- Nas threads, isso é automático: todas compartilham o espaço de endereçamento do processo.

```
typedef struct {
    sem_t sem_lock;
    int   next_row;
} SharedState;

Color *frameBuffer = mmap(NULL, fb_size,
        PROT_READ | PROT_WRITE,
        MAP_SHARED | MAP_ANONYMOUS, -1, 0);

SharedState *shared = mmap(NULL, sizeof(SharedState),
        PROT_READ | PROT_WRITE,
        MAP_SHARED | MAP_ANONYMOUS, -1, 0);
```

Sincronização

- A única região crítica é pegar a próxima linha: `y = next_row++`.
- **Processos:** semáforo POSIX **binário**, compartilhado entre processos: `sem_init(&sem, pshared=1, valor=1)`. Na prática, funciona como um mutex entre processos.
- **Threads:** `pthread_mutex_t`.
- **Mutex × semáforo:** o mutex tem dono (só quem trava destrava); o semáforo é um contador (qualquer um pode dar `sem_post`). Com valor inicial 1, os dois garantem exclusão mútua.
- **Por que semáforo nos processos:** o `pshared=1` do `sem_init` é o jeito direto de compartilhar entre processos. Um mutex também funcionaria, mas exigiria o atributo `PTHREAD_PROCESS_SHARED`.
- **Custo:** 4320 entradas na região crítica em todo o programa, desprezível frente a cerca de 11 s de cálculo.

```
// processos
sem_wait(&shared->sem_lock);
int y = shared->next_row++;
sem_post(&shared->sem_lock);

// threads
pthread_mutex_lock(&q->mutex);
int y = q->next_row++;
pthread_mutex_unlock(&q->mutex);
```

Implementação com threads

- A thread principal lê o número de threads da linha de comando, cria as threads com `pthread_create()` e espera todas com `pthread_join()`.
- Todas recebem um ponteiro para a mesma `WorkQueue`: fila, mutex e framebuffer.
- **Memória compartilhada sem esforço:** o framebuffer é um `malloc` comum, visível a todas as threads do processo.
- O mesmo laço da fila da versão com processos, com mutex no lugar do semáforo.

```
typedef struct {
    int next_row;
    pthread_mutex_t mutex;
    int width, height;
    Color *frameBuffer;
} WorkQueue;

for (t = 0; t < num_threads; t++)
    pthread_create(&threads[t], NULL,
                   threadWorker, &queue);
for (t = 0; t < num_threads; t++)
    pthread_join(threads[t], NULL);
```

Container e limite de CPU

- `container/Dockerfile`: Ubuntu 24.04 + `build-essential`; copia o código, roda `make` e coloca os programas no `PATH`.
- Não é preciso instalar nada no sistema, só ter o Docker.
- `--cpus=N` **não** reduz os núcleos visíveis (o container continua vendo 8). Ele limita o **tempo de CPU** do grupo: N × 100 ms de CPU a cada janela de 100 ms (cota do escalonador CFS via cgroups).
- Quando a cota acaba, todos os processos/threads do container ficam pausados até a próxima janela (*throttling*).

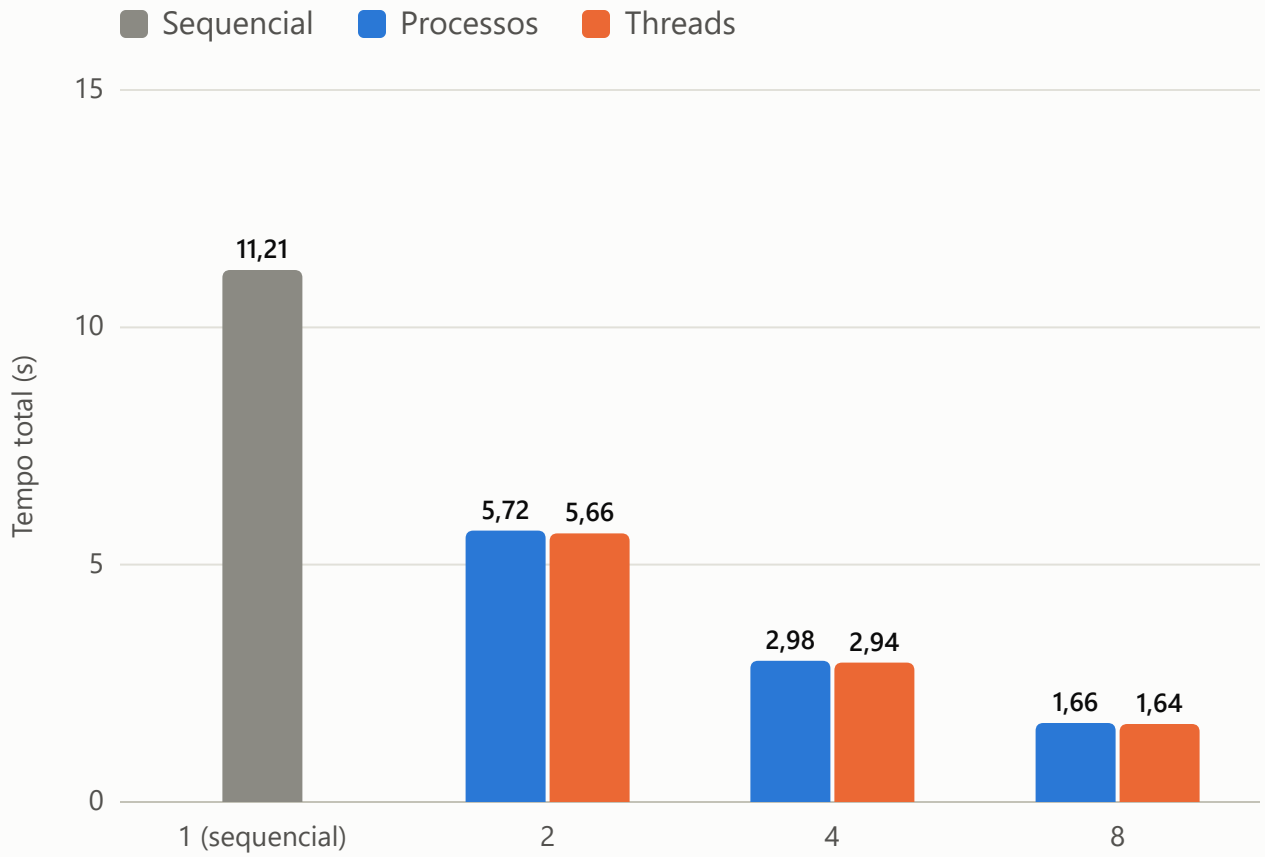
```
docker build -t mandelbrot -f container/Dockerfile .

docker run --rm --cpus="2" mandelbrot \
       processos 4 imagem.ppm 480
```

Metodologia dos experimentos

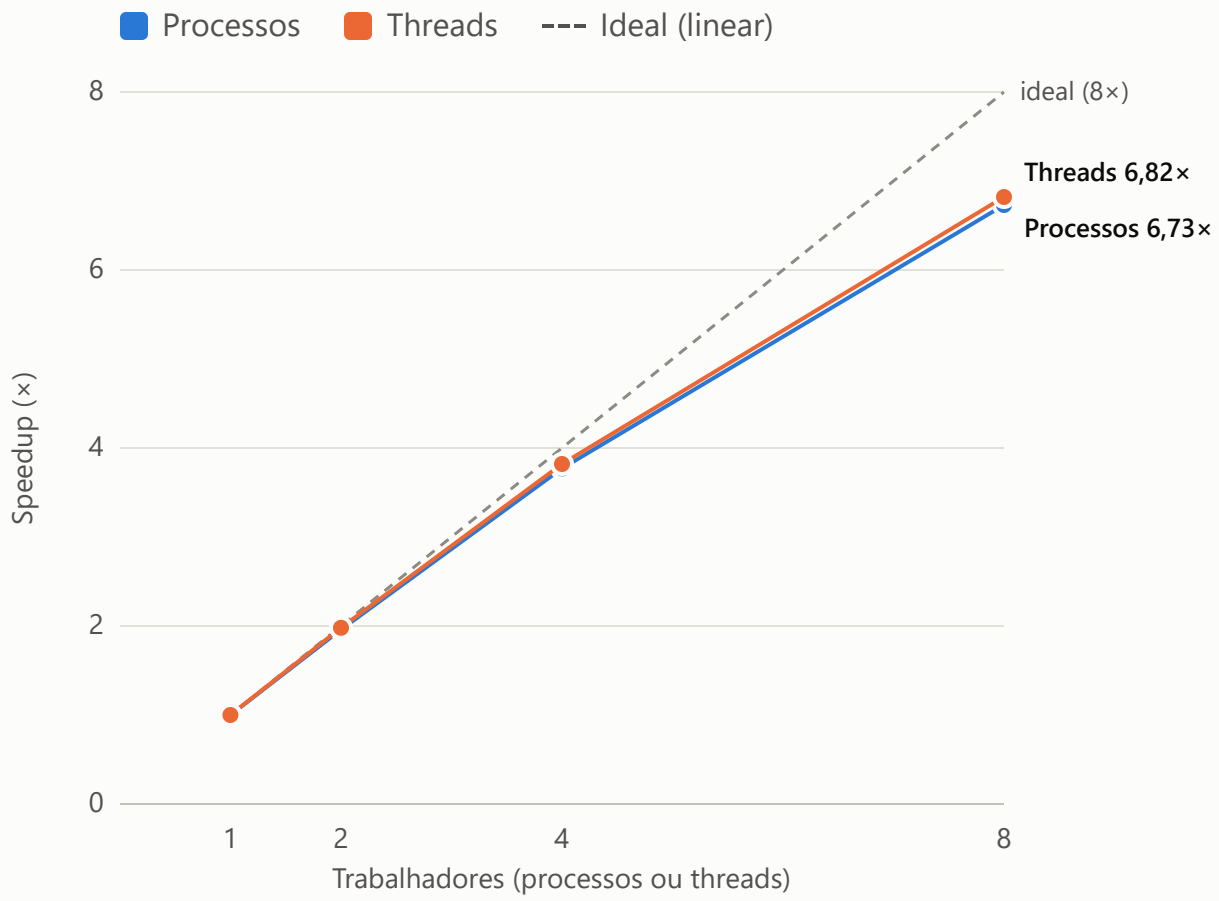
- **Máquina:** Intel Core i5-12600K, WSL2 (Ubuntu 22.04) com 8 CPUs lógicas; gcc 11.4 `-O3`; Docker 29.9.2.
- **Entrada:** `factor=480` (7680 × 4320); **média de 5 execuções** por configuração.
- **Fora do container** (`testes/executar_testes.sh`): sequencial; processos e threads com 2, 4 e 8.
- **Dentro do container** (`testes/executar_testes_docker.sh`): sequencial, processos e threads com 2, 4 e 8, para cada limite `--cpus` = 1, 2 e 4.
- **Speedup** = tempo do sequencial ÷ tempo da versão paralela (dentro do container, o sequencial com o mesmo limite).
- **Corretude:** cada imagem é comparada byte a byte com a do sequencial: **28 de 28 execuções idênticas**.
- Resultados congelados em `testes/resultados/final_*.csv`.

Gráfico 1: tempo de execução (fora do container)



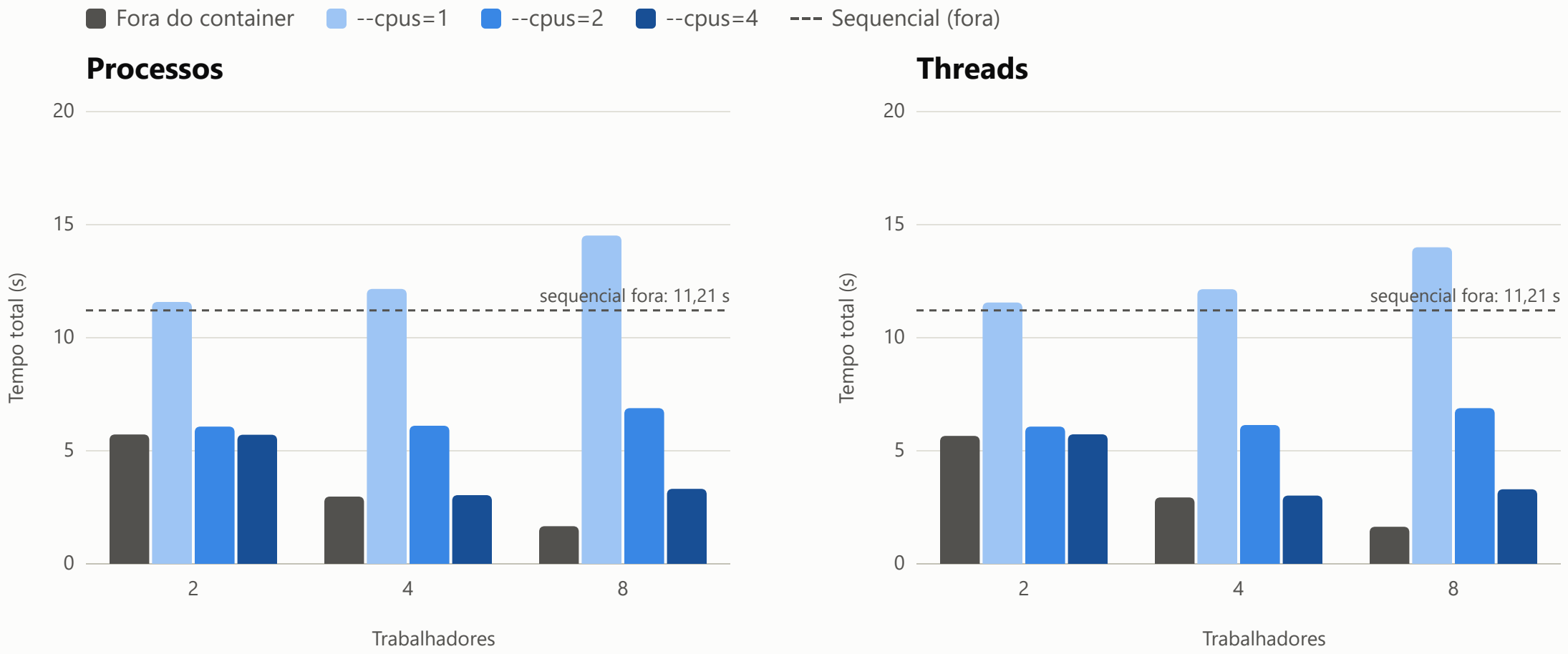
- Dobrar os trabalhadores quase divide o tempo pela metade até 4.
- De 4 para 8, o ganho continua, mas menor.
- Processos e threads ficam praticamente empatados.

Gráfico 2: speedup (fora do container)



- 2 trabalhadores: 1,96× e 1,98×, quase ideal.
- 4: 3,77× e 3,82×.
- 8: 6,73× e 6,82×, longe dos 8×.
- A curva se afasta da ideal conforme cresce o número de trabalhadores. A explicação está nas conclusões.

Gráfico 3: fora × dentro do container



Dentro do container: speedup por limite de CPU

Speedup (proc / thr)	2	4	8
--cpus=1	0,98× / 0,98×	0,93× / 0,93×	0,78× / 0,81×
--cpus=2	1,90× / 1,89×	1,88× / 1,87×	1,67× / 1,67×
--cpus=4	1,96× / 1,95×	3,68× / 3,70×	3,37× / 3,39×

Speedup em relação ao sequencial com o mesmo limite de CPU.

- **O ganho é limitado pelas CPUs disponíveis**: nunca passa do limite, e o melhor caso é um trabalhador por CPU.
- **Mais trabalhadores que CPUs piora o tempo**: com 1 CPU, 8 trabalhadores levam 28% (processos) e 23% (threads) a mais que o sequencial.
- **O próprio limite tem custo**: com 2 trabalhadores e 2 CPUs, o speedup é 1,90×, contra 1,96× fora do container.

Comparação entre as três versões

	Sequencial	Processos	Threads
Fluxos de execução	1	N processos (`fork`)	N threads (`pthread_create`)
Memória da imagem	`malloc`	`mmap(MAP_SHARED)` , explícita	`malloc` , compartilhada naturalmente
Sincronização	nenhuma	semáforo binário (`pshared=1`)	`pthread_mutex_t`
Espera pelo fim	-	`waitpid`	`pthread_join`
Tempo com 8 trabalhadores	11,21 s	1,66 s (6,73×)	1,64 s (6,82×)
Imagem gerada	idêntica byte a byte nas três		

Processos e threads empatam porque toda a comunicação é pegar a próxima linha (4320 vezes) e escrever em memória compartilhada sem lock. O custo extra do `fork` e do semáforo entre processos não aparece frente a cerca de 11 s de cálculo.

Lei de Amdahl: qual speedup seria possível?

- Todo programa tem uma parte que **só roda em sequência** (fração `f`) e uma parte **paralelizável** (`1 − f`). Com `N` trabalhadores, só a parte paralelizável fica `N` vezes mais rápida:

$$S(N) = 1 / (f + (1 - f) / N)$$

- A parte serial limita o ganho: mesmo com infinitos trabalhadores, `S ≤ 1 / f`. Por isso o teórico fica **sempre abaixo de N**.
- **Aqui, a parte serial é tudo que não é cálculo**: alocação do framebuffer, gravação da imagem e liberação da memória, feitas só pelo processo/thread principal: `f = (Total − Cálculo) ÷ Total = 0,0279 s ÷ 11,2071 s ≈ 0,00249` (0,249%), com teto `1 / f ≈ 401×`
- Para 8 trabalhadores: `S(8) = 1 / (0,00249 + 0,99751 / 8) ≈ 7,863×`.

Trabalhadores	Amdahl (teórico)	Medido (proc / thr)
2	1,995×	1,96× / 1,98×
4	3,970×	3,77× / 3,82×
8	7,863×	6,73× / 6,82×

O teórico fica um pouco abaixo de N (1,995× e não 2×) por causa da parte serial. Até 4 trabalhadores, o medido fica perto do teórico. Com 8, fica bem abaixo: **a diferença não é explicada pela parte serial**, que é mínima. A causa está no hardware (próximo slide).

Conclusões

- **A parte serial é mínima** (0,25%: alocação, gravação e liberação). A Lei de Amdahl permitiria 7,86× com 8 trabalhadores, e medimos 6,82×: **a perda não vem da parte serial**.
- **O speedup não é linear por causa do hardware.** As 8 CPUs lógicas do WSL não são 8 núcleos iguais e independentes: há hyperthreading (dois fluxos dividindo um núcleo) e núcleos P/E no i5-12600K, e a frequência de turbo cai quando mais núcleos estão ativos.
- **A divisão dinâmica funciona:** com a fila de linhas, os trabalhadores terminam juntos mesmo com custo por linha muito desigual.
- **Processos ≈ threads:** como os dois usam memória compartilhada e uma região crítica mínima, o mecanismo quase não pesa. A diferença de custo entre eles (criação, troca de contexto) desaparece frente ao cálculo.
- **Docker:** sem limite apertando, o container tem desempenho praticamente igual à execução direta. O `--cpus` limita o ganho ao número de CPUs da cota e tem um pequeno custo próprio (pausas de throttling).
- **Mais trabalhadores que núcleos degrada o desempenho:** mais trocas de contexto e cota consumida por fluxos que dividem o mesmo núcleo. O número de trabalhadores deve acompanhar a CPU disponível.